

Python Day 19

Note: Today's topic:- Instances, State & Higher Order Functions

What you're learning Today (Big picture)
By the end of Day 19, you should be able to:

- Understand object instances and state
- Use higher order functions
- Control programs using keyboard events
- Build interactive games with turtle
- Think like a game developer, not just a coder

Instance & State (Very Important Concept)

• Instance

An instance is a specific object created from a class

```
from turtle import Turtle
tim = Turtle()
```

- Turtle → class
- tim → instance of the class

you can create multiple instances from the scene class:

```
tom = Turtle()
jim = Turtle()
```

Each turtle is independent state

(*) State

State = (current values of an object's attributes)

Example:

- Position (x, y)
- Direction (heading)
- Color
- Speed

```
tim.forward(100)
```

```
tim.left(90)
```

(forward, left, etc.) state is changed
jim's state changes after every method call
Think: "Same turtle, different moved position"

Attributes vs Methods (Quick Revision)

Type	Meaning	Example
Attribute	variable belonging to Object	tim.color
Method	function belonging to object	tim.forward

```
tim.color("red") # method
```

```
tim.Speed(5) # method
```


Higher Order Functions (Core Python IDEA)

• Definition

A Higher Order function is a function that:

- takes another function as input
Or
- returns a function

(*) Sample Example

```
def add(a, b):
```

```
    return a + b
```

```
def calculator(func, a, b):
```

```
    return func(a, b)
```

```
calculator(add, 3, 5)
```

Here:

- calculator() is higher order
- add is passed without parentheses

(x) wrong:

```
calculator(add(), 3, 5)
```

(✓) correct:

```
calculator(add, 3, 5)
```

Turtle Event Listeners (Most Important Part)

- why Event Listeners?

Instead of running code line by line we:

- wait for user input
- then execute functions

This is how games work

listen() and Onkey() (Keyboard Control)

- Step-by-step step

```
from turtle import Screen, Turtle
screen = Screen()
tim = Turtle()
```

Screen.listen()

- listen() → tells program to start listen to keyboard events

Binding Key to Functions

```
def move_forward():
    tim.forward(10)
Screen.onkey(move_forward, "w")
```

Important Rules:

- Do NOT add parentheses to function
- Key must be a string

Common Key Used

Key	Meaning
"w"	forward
"s"	backward
"d"	right
"space"	reset

Turtle Movement Functions

def move_forward():
 tim.forward(10)

def move_backward():
 tim.backward(10)

def turn_left():
 tim.left(10)

def turn_right():
 tim.right(10)

Bind them:

Screen.onkey(move_forward, "w")

Screen.onkey(move_backward, "s")

Screen.onkey(turn_left, "a")

Screen.onkey(turn_right, "d")

Reset/clear Screen

def clear_screen():

tim.clear()

tim.penup()

tim.home()

tim.pendown()

Bind it:

Screen.onkey(clear_screen, "c")

Difference Between clear() and reset()

Method

what it does

clear()

clears drawing only

reset()

clear + reset turtle state

Exitonclick (Program Ending)

Screen.exitonclick()

- keeps window open
- closes only when clicked

Elec - A - Sketch Mini project (Exam - Ready)

- Logic
- Use WASD keys
- Draw while moving
- clear with c

Full Structure (Must Memorize flow)

```
from turtle import Turtle, Screen
```

```
tim = Turtle()
```

```
screen = Screen()
```

```
def forward():
```

```
    tim.forward(10)
```

```
def backward():
```

```
    tim.backward(10)
```

```
def left():
```

```
    tim.left(10)
```

```
def right():
```

```
    tim.right(10)
```



```
def clear():
    tim.clear()
    tim.penup()
    tim.home()
    tim.pendown()
```

```
Screen.listen()
Screen.onkey(forward, "w")
Screen.onkey(backward, "s")
Screen.onkey(left, "a")
Screen.onkey(right, "d")
Screen.onkey(clear, "c")
```

```
Screen.exitonclick()
```

Common Mistakes

(X) using parentheses in onkey()
Screen.onkey(move(), "w") # wrong

(✓) correct
Screen.onkey(move, "w")

(X) forgetting listen()
Screen.onkey(move, "w") # won't work

(✓) Always
Screen.listen()

One line Summary

Day 19 teaches how to control object instances using higher order functions and keyboard event listeners to build interactive programs and games using Tk